ISTQB® OFFICIAL SYLLABUS REFERENCE GUIDE

Chapter 2: Prompt Engineering for Effective Software Testing

Syllabus Module Focus Allocation: 365 minutes

2.1 Effective Prompt Development

- **The 6-Component Prompt Blueprint:**
  - *Role:* Persona or perspective the model assumes (e.g., Test Automation Engineer) to dictate tone and responsibility.
  - *Context:* Background details detailing the test object, target application architecture, or domain framework.
  - *Instruction:* Concise, imperative directives outlining the precise test task to be completed.
  - *Input Data:* Relevant assets provided to help process the task (user stories, code bases, specs, wireframes).
  - *Constraints:* Boundaries, negative guidelines, or explicit structural restrictions to avoid hallucinations.
  - *Output Format:* Defined template layout for responses (Gherkin format, Markdown table, valid JSON).
- **Core Prompting Techniques:**
  - *Prompt Chaining:* Breaking a complicated test process into smaller sequential prompts. The output of each phase is verified/cleaned before feeding downstream.
  - *Few-Shot Prompting:* Providing a few (2+) structural input-output examples directly inside the prompt to anchor model patterns (vs zero-shot/no examples or one-shot/one example).
  - *Meta Prompting:* Instructing the LLM to generate, edit, or recursively optimize its own testing prompts based on high-level objectives.
- **Interface Elements:**
  - *System Prompt:* Background command set configured by engineers to govern long-term behavior, operational parameters, and rules for a session (hidden from end-user).
  - *User Prompt:* Dynamic, explicit inputs or questions added interactively by the tester per conversation turn.

2.2 Applying Prompt Engineering to Test Tasks

- **Test Analysis:** Identifying flaws/ambiguities in requirements, generating test conditions, or mapping risk-based priorities.
- **Test Design & Implementation:** Formulating functional cases, synthesizing data-privacy preserving production-like data, or generating automated test scripts across frameworks.

- **Regression Testing & Monitoring:** Building keyword-driven suites, updating locators to self-heal dynamic UI/API specification drift, and clustering regression results to extract summary trends.

2.3 Evaluation and Refinement

- **AI Quality Evaluation Metrics:** *Accuracy* (overall correctness against standards), *Precision* (correct anomaly identification), *Recall* (coverage across partitions), *Relevance* (contextual fit), *Diversity* (edge-case exposure), *Execution Success Rate* (compilable script %), and *Time Efficiency* (gains vs manual time).